

```
@page { margin: 14mm 12mm; } body { font-family: "DejaVu Sans",
Arial, sans-serif; font-size: 10.5pt; line-height: 1.34; color: #1a2733;
max-width: 100%; margin: 0; } h1 { font-size: 19pt; color: #12324b;
border-bottom: 2px solid #12324b; padding-bottom: 3px; } h2 { font-
size: 14pt; color: #12324b; border-left: 4px solid #12324b; padding-
left: 7px; margin-top: 16px; } h3 { font-size: 12pt; color: #234;
margin-top: 12px; } table { border-collapse: collapse; width: 100%;
margin: 8px 0; font-size: 9pt; } th, td { border: 1px solid #b7c2cc;
padding: 3px 6px; text-align: left; vertical-align: top; } th {
background: #12324b; color: #fff; } tr:nth-child(even) td {
background: #f2f5f7; } code { background: #eef1f4; padding: 1px
3px; border-radius: 3px; font-size: 8.8pt; } pre { background:
#0f1b26; color: #dfe7ee; padding: 8px 10px; border-radius: 5px; font-
size: 8.4pt; overflow-x: auto; white-space: pre-wrap; } pre code {
background: transparent; color: inherit; }
```

VCS_PORTABILITY_NOTES

O-RAN VIP Suite — VCS Portability & Scheduling Notes

Author: AVIK MAJUMDAR **Project:** AVIK VIP FACTORY — O-RAN VIP Suite **2026-08-09**

Scope: three portability items raised for the native-UVM lanes (Lane-2B suite + uvm_reg RAL) when moving from the Verilator 5.050 reference run to VCS/Questa/Xcelium. Item 1 is a code fix; item 2 is a scheduling analysis; item 3 is the harness-verification status.

1. Package name resolution — import leaf packages directly, never through a re-export chain

Rule applied: every leaf package (<slug>_uvm_pkg, oran_ral_pkg, ecpri_uvm_pkg) is compiled and **imported directly** in the top module. There is no import X::*; export X::*; re-export chain anywhere in the suite (verified: `grep -rn "export .*:;" lane2/` returns nothing). VCS will not resolve a typedef or class through a re-export chain, so the design avoids them entirely — `oran_uvm_tb_top.sv` lists all 13 import <slug>_uvm_pkg::*; lines explicitly, and `oran_ral_tb_top.sv` imports `oran_ral_pkg::*` directly.

Root-cause fix delivered (double-UVM definition): the original `ecpri.f` compiles the Accellera source `$UVM_HOME/src/uvm_pkg.sv`. That filelist is correct **only** for Verilator 5.050, which has no built-in UVM. On VCS the run also passed `-ntb_opts uvm-1.2` (VCS’s own UVM), so `uvm_pkg` was defined **twice** — and VCS then fails to resolve `uvm_*` names through the (now ambiguous) import chain. This presents exactly as “won’t resolve any name — typedef or class — through the chain.”

The fix is a strict split of filelists by venue:

Venue	UVM source	Filelist	Contains uvm_pkg.sv?
Verilator 5.050	Accellera (\$UVM_HOME/src)	build_suite_uvm.sh / build_ral.sh (explicit)	yes (Verilator has no built-in UVM)
	vendor (-	suite_commercial.f / ral_commercial.f	

VCS	ntb_opts uvm-1.2)	/ecpri_commercial.f	no
Xcelium	vendor (-uvm)	same*_commercial.f	no
Questa	vendor (-L uvm)	same*_commercial.f	no

Never compile the Accellera uvm_pkg.sv **and** enable a vendor UVM library in the same VCS/Questa/ Xcelium run. Use run_commercial.sh <sim> (suite + RAL) or run_lane2b.sh <sim> (ecpri) — both now point at the *_commercial.f filelists and the vendor built-in UVM only.

2. VCS-vs-Verilator event-scheduling / sampling race — analysis

Verified-passing path is race-free by construction. In every Lane-2B component scoreboard and in the RAL, checking and coverage sampling happen at the **transaction level**, inside the analysis write() function:

```
virtual function void write(<slug>_item it);    // called by driver's
ap.write(req)
    cv_<field> = it.<field>;                    // blocking, same
    call                                         //
    cg.sample();                               // explicit sample,
    zero-time
    ... scoreboard checks ...
endfunction
```

This is a pure function call chain (driver ap.write() → imp → scoreboard write()), executed in zero simulation time in the Active region. It has **no** dependency on the NBA/Observed region, no @(posedge clk), and no clocking-block sampling — so it evaluates identically under VCS's and Verilator's event schedulers. There is **no sampling race** in the path that produces the 1523/1523 PASS evidence. The RAL front-door accesses are likewise serialized through the sequencer (get_next_item/item_done ping-pong), so read-after-write ordering is guaranteed by UVM sequencer arbitration, not by clock timing — again race-free.

The only clock-sampled constructs are the four concurrent SVA in ecpri_if.sv (@(posedge clk) valid |-> ...). In the current build the interface is **transaction-decoupled**: the driver publishes the item to the scoreboard via the analysis port rather than wiggling vif pins, so valid is never asserted and these properties are **vacuous, not racy**. No signal is both NBA-driven and clock-sampled anywhere, so no race is reachable in the reference run.

Race-immune pattern for exercising the SVA on a licensed venue. When the concurrent assertions are wired to real interface activity (a commercial-sim enhancement), drive the interface with **non-blocking assignments synchronized to the sampling edge** so the assertion samples settled values in the Observed region (one delta after the driving NBAs resolve):

```
// in ecpri_driver.run_phase – race-immune driving
forever begin
    seq_item_port.get_next_item(req);
    @(posedge vif.clk);                // align to the SVA sampling
clock
    vif.valid      <= 1'b1;            // NBA: resolves in NBA region,
sampled in Observed region
    vif.version    <= req.version;
    vif.msg_type   <= req.msg_type;
    vif.seq_id     <= req.seq_id;
    vif.payload_size <= req.payload_size;
```

```

        ap.write(req);
        seq_item_port.item_done();
    end
    // (get vif in build_phase: uvm_config_db#(virtual
ecpri_if)::get(this,"","vif",vif);)

```

Using blocking (=) assignments at/around the clock edge is what would create a VCS/Verilator race (Active-region write vs. concurrent-assertion sample); NBA driving avoids it on both simulators. Note that `p_seq_inc (valid ##1 valid |-> seq_id == $past(seq_id)+1)` additionally requires a monotonic-`seq_id` sequence to be non-vacuously true — use a directed incrementing sequence for that assertion rather than the free-random item. This is a venue-side coverage enhancement, independent of the race question, which is settled: **the reference run has no sampling race.**

3. Harness fix — verification status

The Verilator-on-UVM harness uses two Verilator-specific accommodations:

- **Force-registration:** `if(<test>::type_id::get() != null) reg_cnt++;` for every package test class in the top module. Verilator elides a bare `void' (type_id::get())`, so the factory would otherwise see zero registered types (“No components instantiated”). On VCS/Quarta/Xcelium the `uvm_component_utils` static registration runs automatically, so this guard is a harmless no-op safety.
- **Explicit run_test from plusarg:** `if($value$plusargs("UVM_TESTNAME=%s",tn)) run_test(tn);`. Standard UVM `run_test()` already reads `+UVM_TESTNAME`; the explicit form is equivalent and portable.

Status: the harness (force-registration + explicit `run_test` + watchdog `$finish`) is **verified via the shell path on Verilator 5.050** — suite 13/13 and RAL 1/1 executed, `** TEST PASSED **`, `UVM_ERROR : 0`, `UVM_FATAL : 0`. **The VCS run is pending** (no VCS license in this environment); the commercial filelists and `run_commercial.sh` / `run_lane2b.sh` are prepared and the double-UVM blocker is removed, so the VCS bring-up is expected to be a clean compile+run against the vendor UVM. Until a VCS log is produced, the VCS column stays **NOT_RUN / source-ready** — never reported as PASS on inference.

Reproduce (commercial venues)

```

cd lane2/lane2b_native/suite && ./run_commercial.sh vcs all 1      #
13 suite tests + RAL, VCS vendor UVM
cd lane2/lane2b_native/suite && ./run_commercial.sh xcelium all 1  #
Xcelium -uvm
cd lane2/lane2b_native/suite && ./run_commercial.sh questa all 1   #
Quarta -L uvm
cd lane2/lane2b_native/ecpri_transport && ./run_lane2b.sh vcs 1    #
single-component ecpri

```

Reference (Verilator 5.050, verified): `make lane2b SIM=verilator5050 && make ral SIM=verilator5050.`